

■ Design Instagram

System Design Interview | Social Media Platform

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	Upload photos/videos; Follow users; News feed (chronological + ranked); Like/comment; Stories (24h expiry); Search
Non-Functional	1B users; 500M DAU; 100M photos/day uploaded; Feed load <200ms; 99.99% uptime; Global CDN
Scale	~1150 uploads/sec; Storage: 100M photos × avg 3MB = ~300 TB/day; Feed generation is the hardest problem

■ High-Level Architecture

- User Service (profiles, follows), Media Service (upload/process), Feed Service (generation), Notification Service
- Media Upload: client gets pre-signed S3 URL → uploads directly to S3 → S3 triggers Lambda → triggers media processing pipeline
- Media Processing: resize to multiple resolutions, transcode video, generate thumbnails → store in S3 → invalidate CDN
- CDN (CloudFront/Akamai) serves all media; origin is S3; 90%+ cache hit rate for popular content

■ News Feed Deep Dive

- **Fan-out on Write (Push):** On post creation, write post to all followers' feed caches immediately; feed read is $O(1)$; write amplification: $100M \text{ followers} \times 1 \text{ post} = 100M \text{ writes}$
- **Fan-out on Read (Pull):** Feed is computed at read time by merging followed users' posts; no write amplification; slow for users with many follows
- **Hybrid (Instagram approach):** Push for regular users; Pull for celebrities (>1M followers) to avoid write storms; Merge at read time
- **Feed Cache:** Redis sorted set per user: {post_id: timestamp}; keep last 200-300 posts; evict oldest; pagination with cursor

■ Data Model

- posts: post_id(PK), user_id, media_url, caption, created_at — PostgreSQL or Cassandra (partition by user_id)
- follows: follower_id, followee_id, created_at — index on both for bidirectional lookups
- feed_cache: Redis sorted set per user; score = timestamp; members = post_ids; ZREVRANGE for paginated feed
- likes/comments: Cassandra (post_id, user_id, timestamp) — high write volume, eventual consistency acceptable

■ Feed Generation Trade-offs

Push (Fan-out on Write)	Pull (Fan-out on Read)
✓ Fast reads $O(1)$	✓ No write amplification
✓ Pre-computed feed	✓ Always fresh data
✓ Works for most users	✓ Slow for many follows
✓ Wasted work for inactive users	✓ High read latency

■ Stories Architecture

- Stories TTL: store in Redis with 24h expiry; also persist in Cassandra for compliance/replay
- Story viewers: ring buffer in Redis; store viewer list per story up to a limit; aggregate count separately
- Story ordering: show unseen stories first, sorted by recency of post; mark seen via Redis SET per user
- Expiry cleanup: Redis TTL handles cache; background job cleans Cassandra; S3 lifecycle policy archives media

■ Search & Discovery

- Hashtag search: Elasticsearch index on hashtags + post metadata; inverted index maps tag $\rightarrow$ post_ids
- User search: Elasticsearch with prefix matching on username/display_name; cache popular searches
- Explore/Recommendations: collaborative filtering (similar users' likes); content-based (visual embedding similarity via ML model)
- Trending: count hashtag usage in sliding 1h window (Redis sorted set ZINCRBY); top-K using min-heap